

10 Regression



10.3 Polynomial Regression

Fitting nonlinear relationships with hand-crafted powers of x — design matrices, overfitting, and bivariate polynomial regression

Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

Core Skills You Will Master in This Section

1

Build nonlinear models by hand-crafting higher powers of x as extra features

2

Understand how the polynomial-regression design matrix grows with degree

3

Use the least-squares formula to compute polynomial regression coefficients b

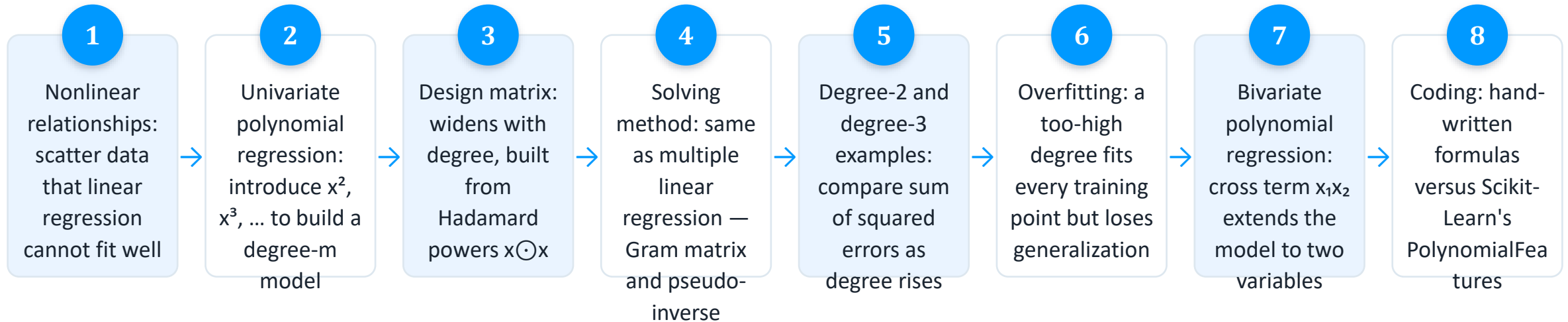
4

Understand underfitting and overfitting, and the trade-off with generalization

5

Extend to bivariate polynomial regression, including cross terms

From Hand-Crafted Nonlinear Features to Overfitting and Beyond



Five Lenses for Every Regression Topic in This Chapter

① Geometric view: the prediction is the orthogonal projection of y onto the span of the regressors

② Algebraic view: design matrix X , Gram matrix $X^T X$, generalized inverse $(X^T X)^{-1} X^T$

③ Optimization view: least squares minimizes the residual sum of squares $\|\epsilon\|^2$

④ Statistical view: residuals, goodness of fit, significance tests — left to Probability and Statistics

⑤ Generalization path: from simple to multiple to nonlinear polynomial regression

⑥ Coding view: hand-written matrix formulas and Scikit-Learn give identical results

⑦ Applications: prediction, trend fitting, and explaining relationships between variables

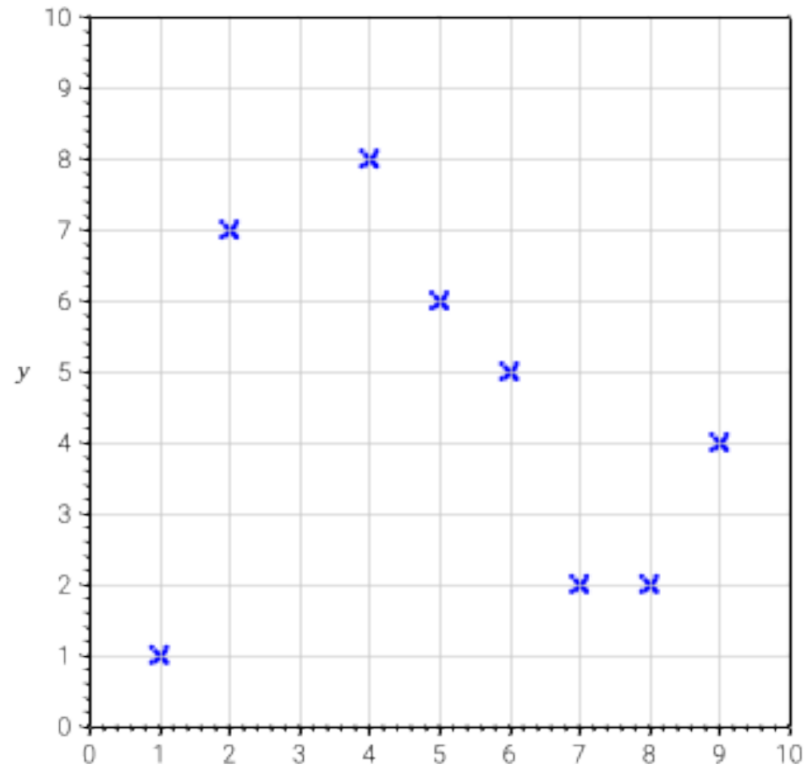
Some scatter data show a curved trend that linear regression cannot fit well

$$y = b_0 + b_1x + \varepsilon \quad (\text{linear regression struggles with curved trends})$$

- We can hand-craft nonlinear features $x^2, x^3, \dots$ expanding to $1, x, x^2, x^3, \dots$
- This idea is exactly what leads to polynomial regression

Figure 1 · Scatter Data Showing a "Nonlinear" Relationship

- 8 points rise then fall as x increases — a straight line cannot follow this curve



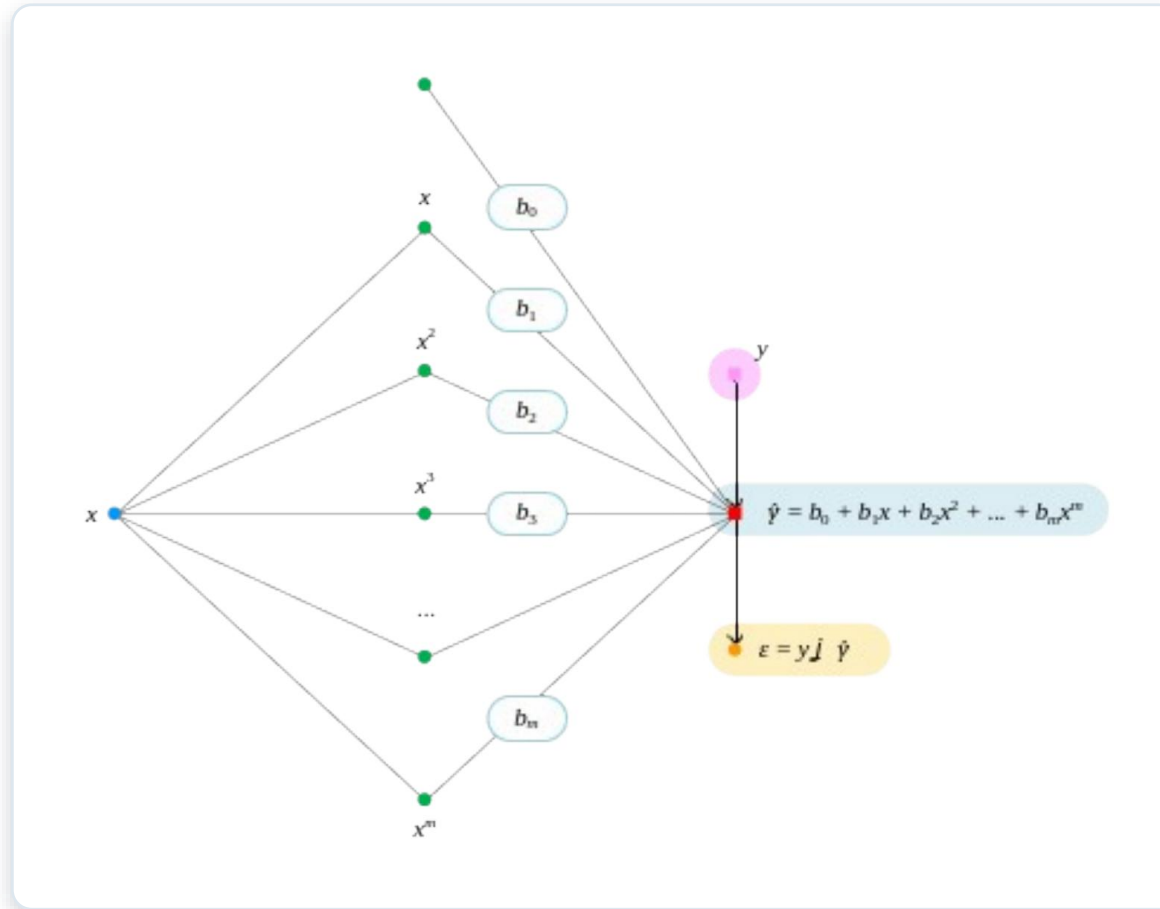
Univariate polynomial regression introduces higher powers of x

$$\hat{y} = b_0 + b_1x + b_2x^2 + \dots + b_mx^m$$

- $b_2x^2, \dots, b_mx^m$ are all "hand-crafted" features; setting $m=1$ recovers simple linear regression
- Simple linear regression is a special case of polynomial regression

Figure 2 · Data Relationship in Univariate Polynomial Regression

- $1, x, x^2, \dots, x^m$ are weighted by b_0 through b_m , summed to $\hat{y}$, subtracted from y for ε



In polynomial regression, each power of x becomes an extra feature column

$$X = [1 \ x \ x \odot x \ \dots \ x \odot x \odot x \odot x \odot x]$$

- $x \odot x$ denotes the element-wise (Hadamard) product
- This is the design matrix for a degree-5 univariate polynomial regression

Figure 3 · Building the Design Matrix in Univariate Polynomial Regression

- As degree grows from 1 to 2 to 5, the design matrix widens with each new power column

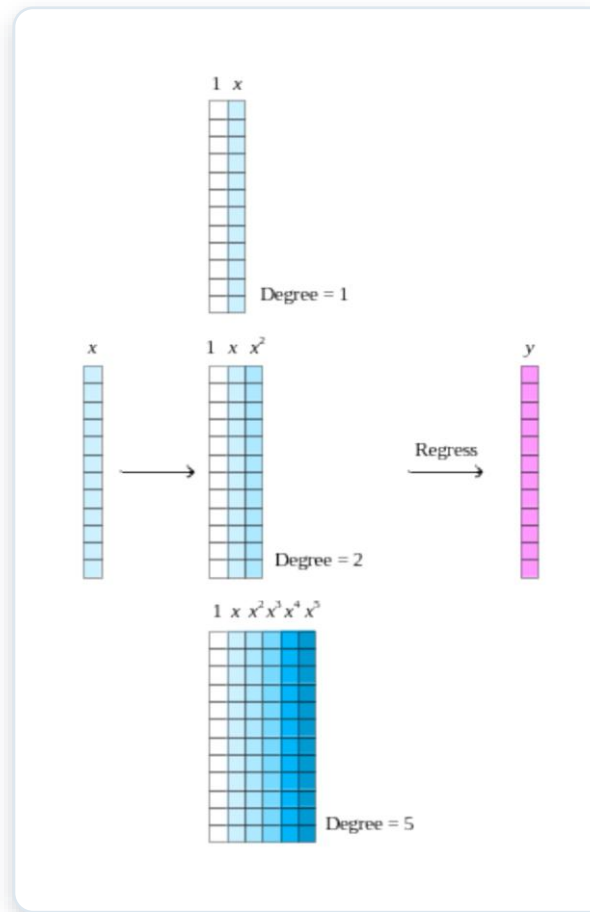
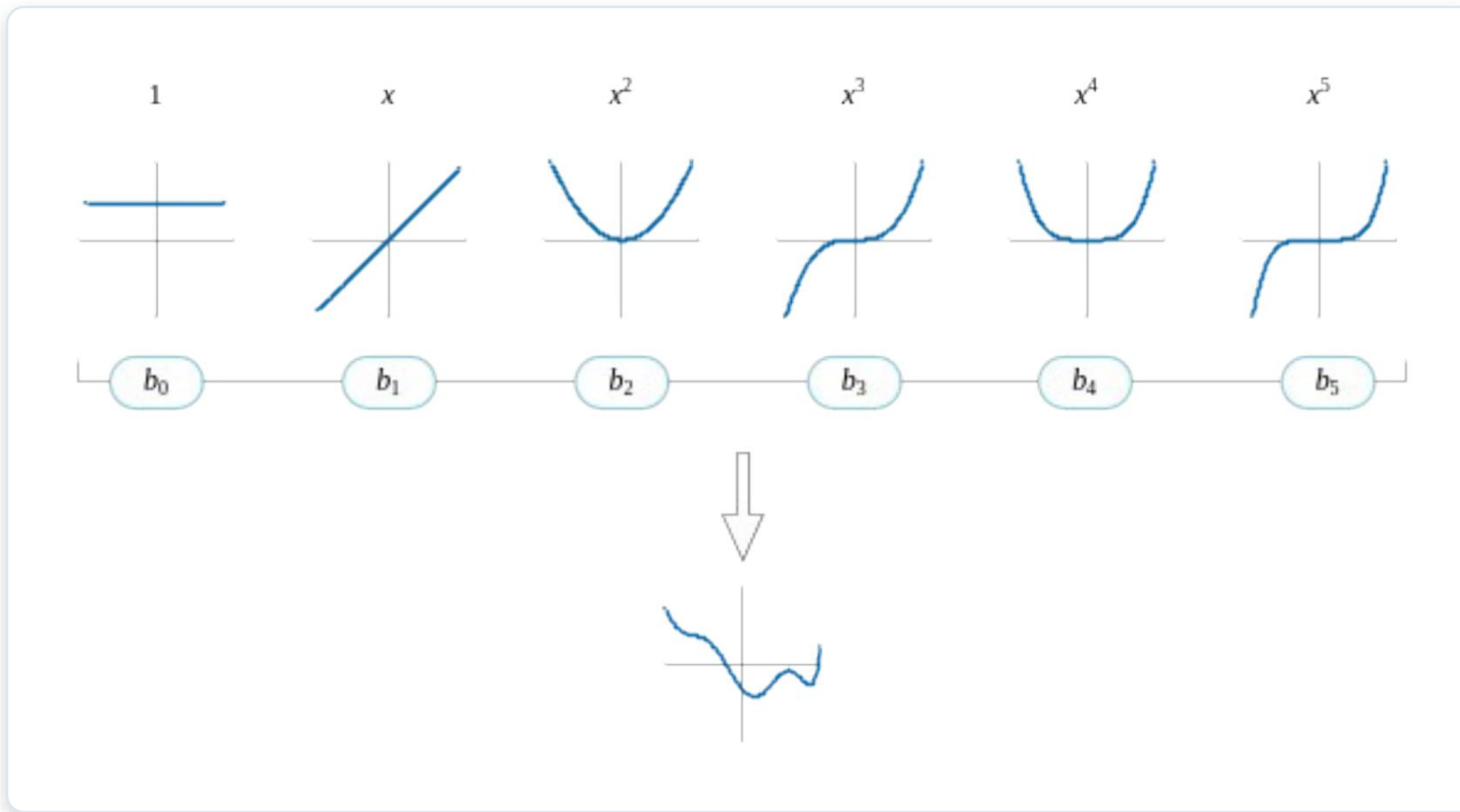


Figure 4 · A Degree-5 Function Viewed as 6 Stacked Curves

- The curves $1, x, x^2, x^3, x^4, x^5$, weighted by b_0 through b_5 , combine into the final curve



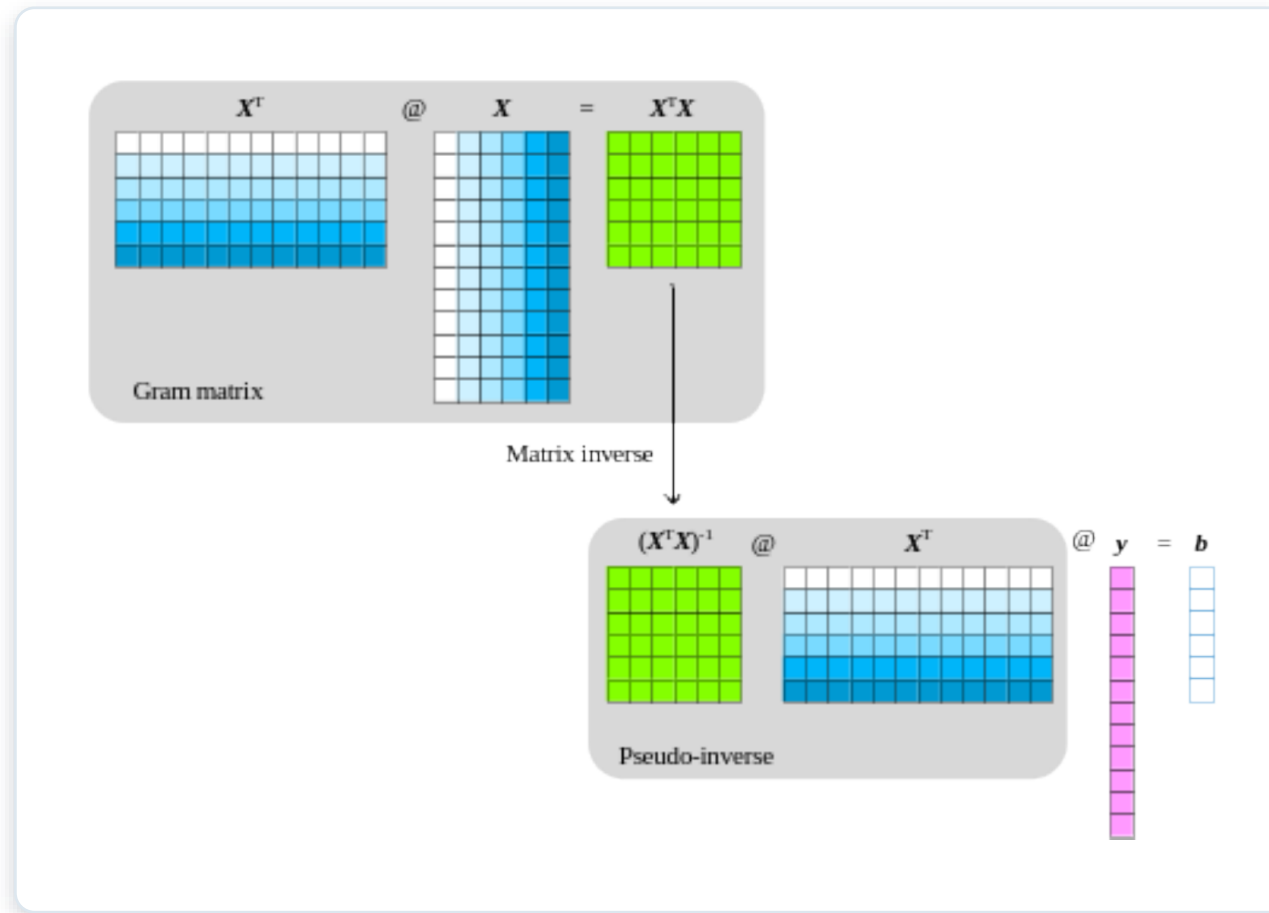
Solving for $\mathbf{b}$ uses exactly the same method as multiple linear regression

$$\mathbf{b} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

- The columns of the design matrix $\mathbf{X}$ also span a multi-dimensional vector space
- Compute the Gram matrix $\mathbf{X}^T \mathbf{X}$ and its inverse, then multiply by $\mathbf{X}^T$ and by $\mathbf{y}$ to get $\mathbf{b}$

Figure 5 · Computing b in Univariate Polynomial Regression

- X^T times X gives $X^T X$; its inverse times X^T is the generalized inverse; times y gives b



As an example, the degree-2 polynomial regression model is

$$\hat{y} = b_0 + b_1x + b_2x^2$$

- Reusing the 8 points from Figure 1: $x=[1,2,4,5,6,7,8,9]^T$, $y=[1,7,8,6,5,2,2,4]^T$
- "Degree 2" means the model includes x^2 ; the highest power sets the polynomial's degree

The design matrix X for degree-2 polynomial regression is

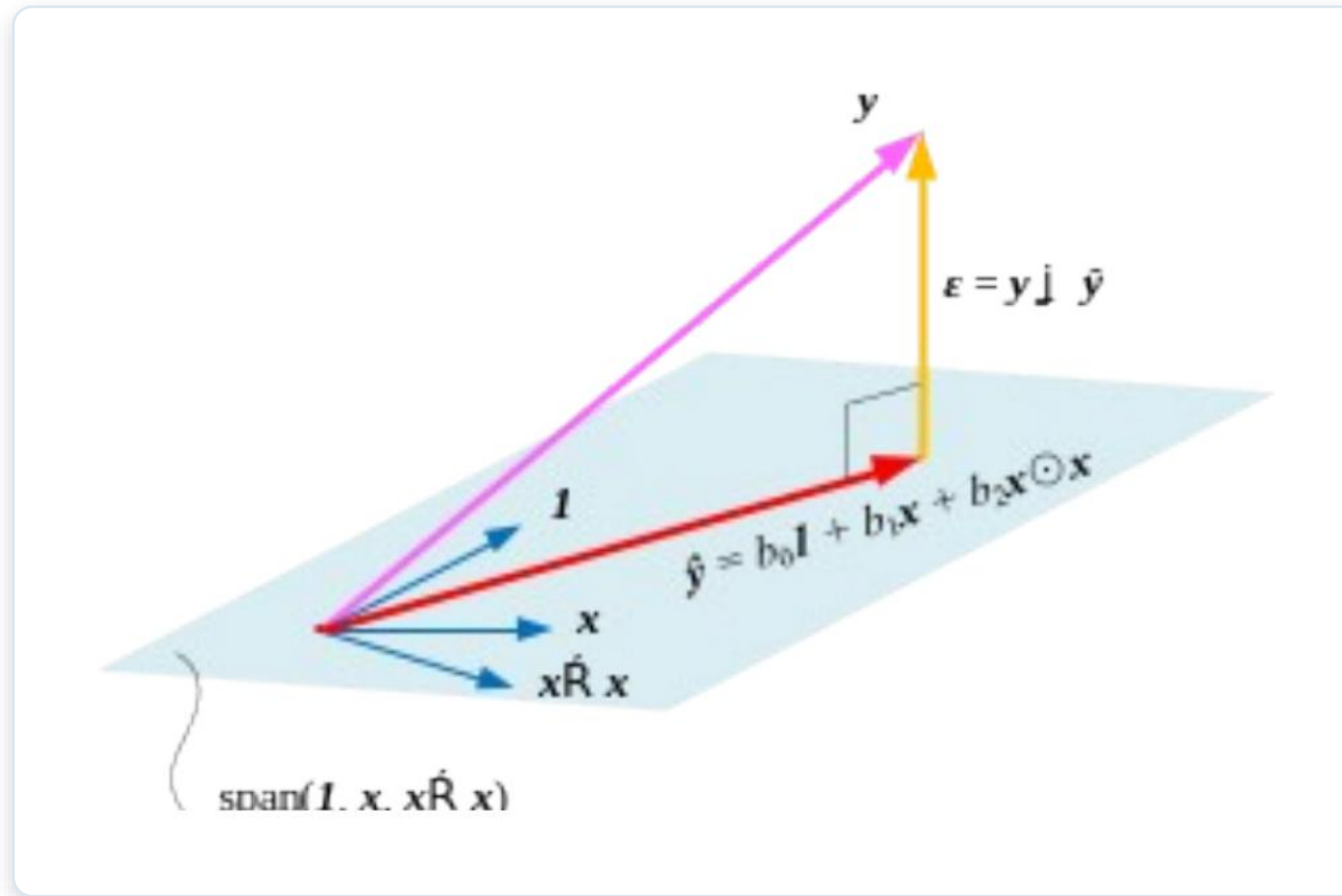
$$X = [1 \ x \ x \odot x] \quad (n=8 \text{ rows}, 3 \text{ columns})$$

- Column 1 is all ones, column 2 is x itself, column 3 is x squared element-wise

PROJECTING y ONTO $\text{SPAN}(1, x, x \odot x)$

Figure 6 · Projecting y onto $\text{span}(1, x, x \odot x)$, Degree-2 Polynomial Regression

- $\hat{y}$ is a linear combination of $1, x, x \odot x$; the error $\varepsilon = y - \hat{y}$ is orthogonal to this hyperplane



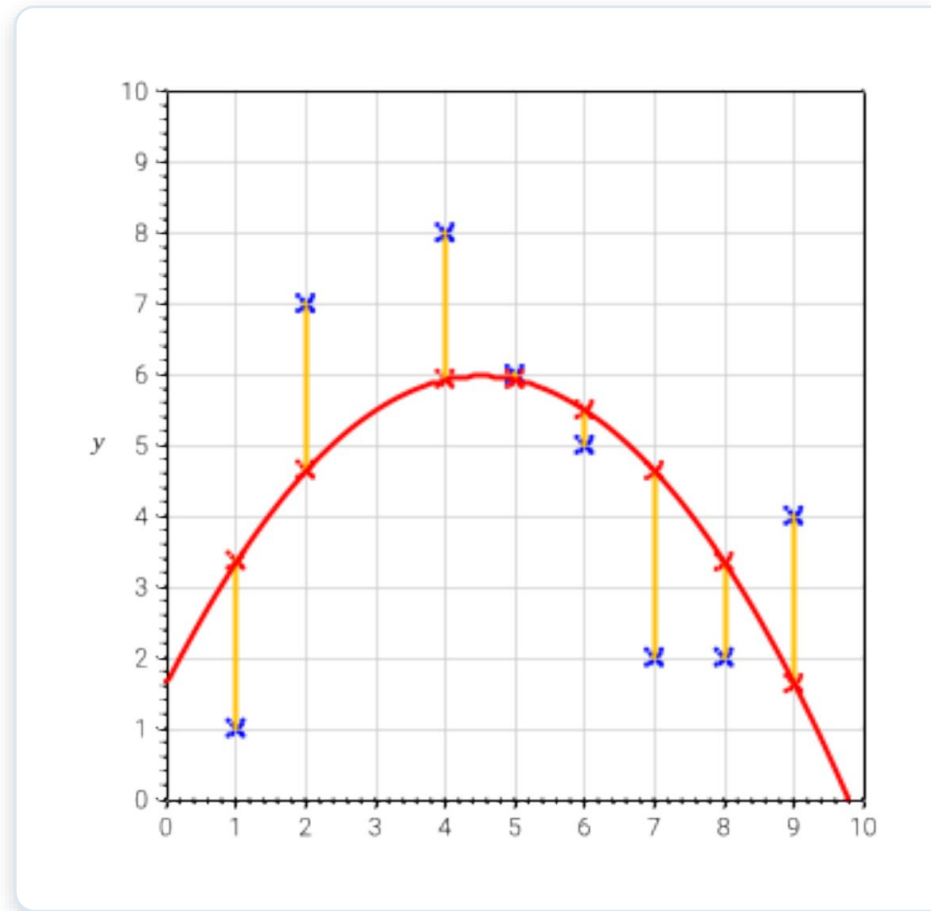
Substituting into the generalized-inverse formula gives the degree-2 model

$$\hat{y} = 1.655 + 1.927x - 0.214x^2$$

- Solving gives $b=[1.655, 1.926, -0.214]^T$; the red parabola in Figure 7 is this curve

Figure 7 · Degree-2 Polynomial Regression

- Orange segments show $\epsilon = y - \hat{y}$; the sum of squared errors $\|\epsilon\|^2 = 30.040$ is still large



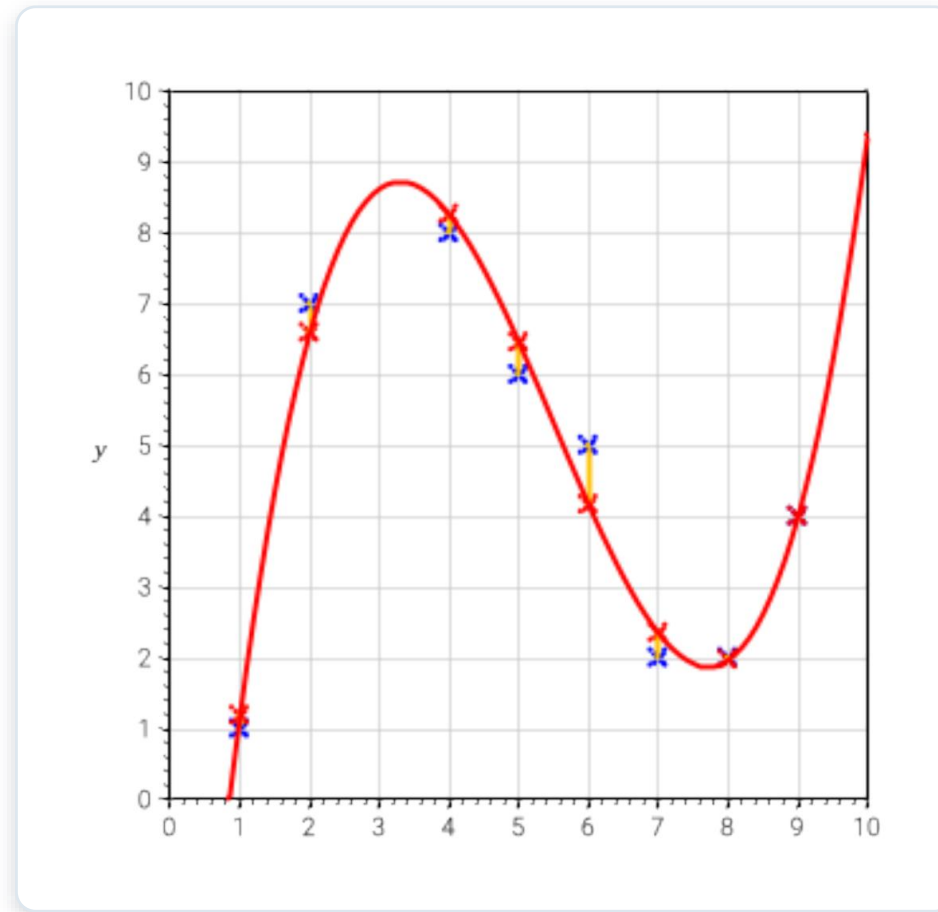
A low degree cannot capture complex relationships — raise it to 3!

$$\hat{y} = b_0 + b_1x + b_2x^2 + b_3x^3$$

- Try writing the degree-3 design matrix X and computing b yourself
- Solving gives $\hat{y} = -8.539 + 12.211x - 2.643x^2 + 0.16x^3$

Figure 8 · Degree-3 Polynomial Regression

- The red curve fits better than Figure 7; $\|\epsilon\|^2=1.302$, much smaller than degree 2



OVERFITTING

Too high a degree can lead to overfitting and poor generalization

Underfitting Like shallow studying — only surface knowledge, missing even the basics

Overfitting Like rote memorization — great on known examples, lost on new variations

Good fit Performs well on both training and test data, with the ability to generalize

DEGREE 8

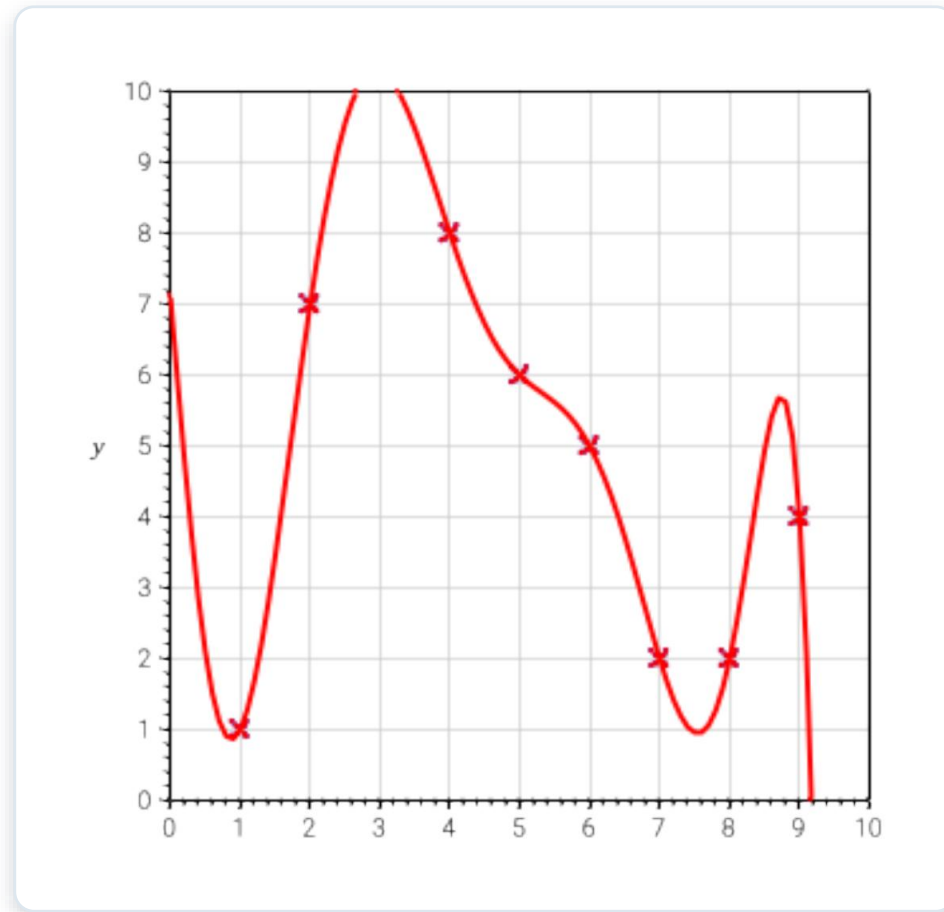
Figure 9 shows the degree-8 polynomial regression result, with equation

$$\hat{y} = 7.123 - 11.430x - 2.828x^2 + 15.342x^3 - 9.463x^4 + \dots$$

- A higher degree fits training data better, but tends to perform worse on new data

Figure 9 · Degree-8 Polynomial Regression

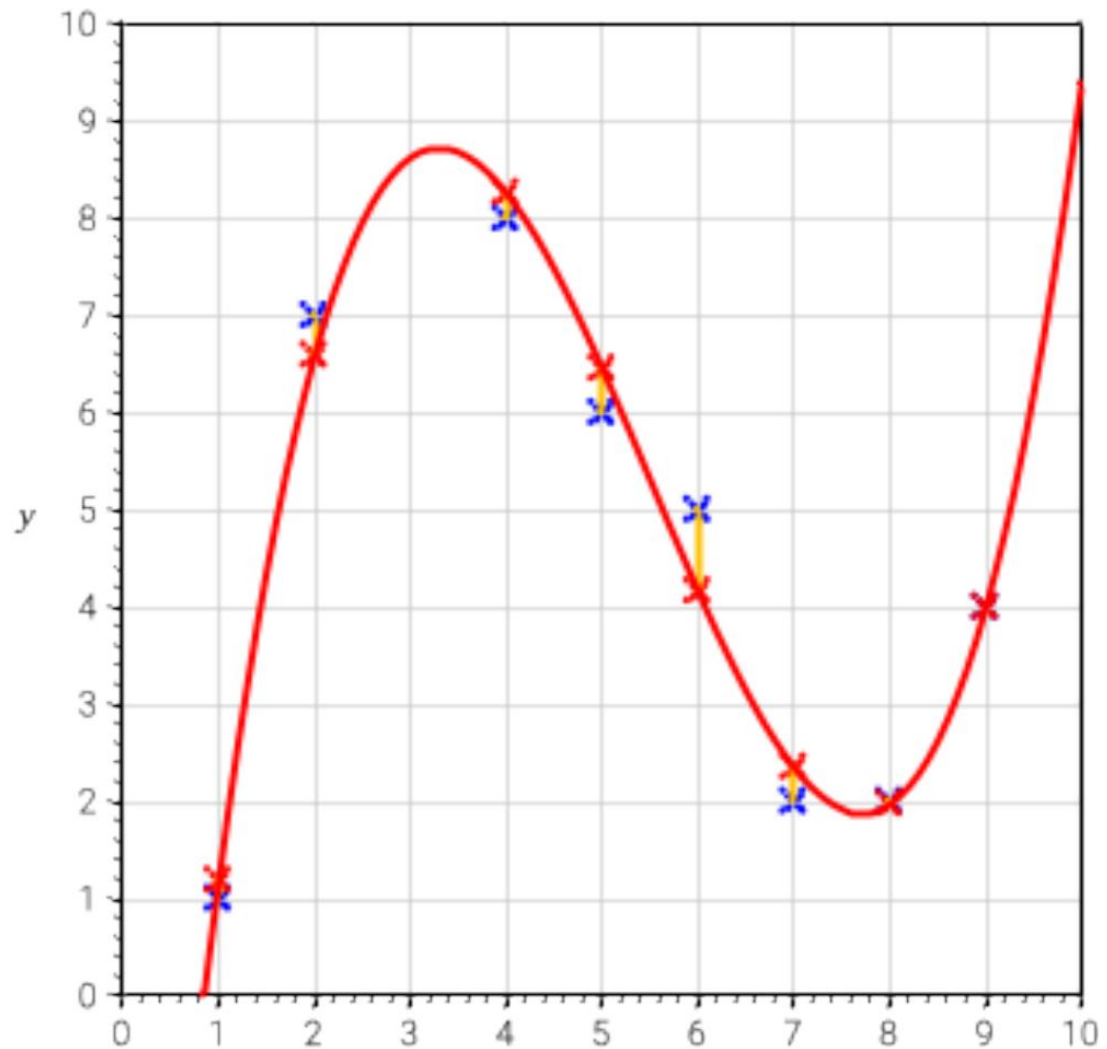
- The red curve passes "perfectly" through every point, yet this is not an ideal model



Solving Degree-3 Polynomial Regression in Python

$b = \text{inv}(X.T@X)@X.T@y$ solves the degree-3 model by hand

- (a) Build one-variable regression data x, y
- (b) Stack design matrix $X=[1, x, x \odot x, x \odot x \odot x]$, 4 columns
- (c) Compute $b=\text{inv}(X^T X)@X^T@y$, the least-squares solution
- (d) Predict $\hat{y}=Xb$ and residual $\varepsilon=y-\hat{y}$ from b
- (e) Plot scatter, fitted curve, and error segments — Figure 8



Fitting Polynomial Features Automatically with Scikit-Learn

Study LA_10_03_02.ipynb: sklearn.preprocessing's PolynomialFeatures auto-generates higher powers

Then sklearn.linear_model's LinearRegression fits the degree-8 model and plots Figure 9, for comparison

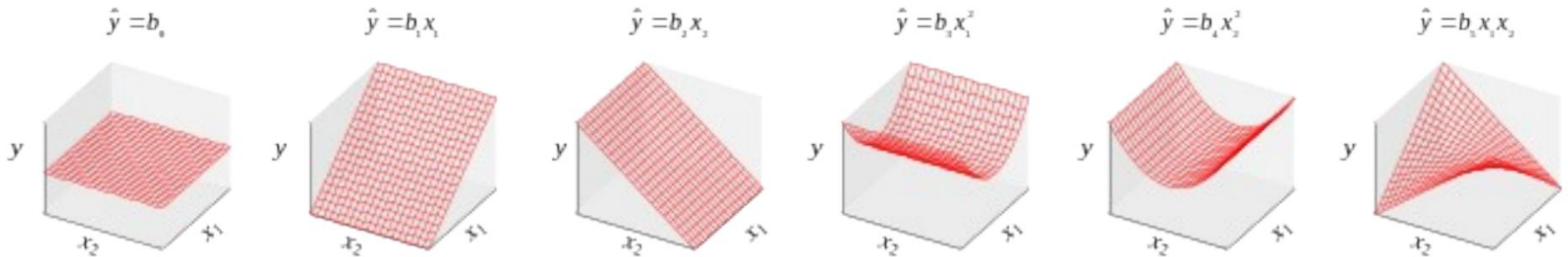
Extending to two variables x_1, x_2 gives bivariate polynomial regression

$$\hat{y} = b_0 + b_1x_1 + b_2x_2 + b_3x_1^2 + b_4x_2^2 + b_5x_1x_2$$

- Besides x_1, x_2 , we add x_1^2, x_2^2 , and the cross term x_1x_2
- This model captures both the linear relationship and the nonlinear interaction between x_1, x_2

Figure 10 · Each Term of a Bivariate Quadratic Polynomial

- The model is a sum of 6 surfaces: the constant, x_1 , x_2 , x_1^2 , x_2^2 , and the cross term x_1x_2



Bivariate quadratic polynomial regression as matrix multiplication

$$\hat{y} = [1 \ x_1 \ x_2 \ x_1 \odot x_1 \ x_2 \odot x_2 \ x_1 \odot x_2] [b_0; \dots; b_5]$$

- The design matrix is $X = [1 \ x_1 \ x_2 \ x_1 \odot x_1 \ x_2 \odot x_2 \ x_1 \odot x_2]$

A higher degree produces even more hand-crafted features

Bivariate cubic polynomial regression brings even more features, including cubic terms

As degree grows, we keep folding in more higher-order terms, as Figure 11 shows

These hand-crafted features let the model fit complex, curved data surfaces

Figure 11 · Terms of Different Degrees in a Polynomial

- Rows and columns of $1, x_1, x_1^2, x_1^3$ and $1, x_2, x_2^2, x_2^3$ multiply into each term

	1	x_2	x_2^2	x_2^3	...
1	1	x_2	x_2^2	x_2^3	
x_1	x_1	$x_1 x_2$	$x_1 x_2^2$	$x_1 x_2^3$	
x_1^2	x_1^2	$x_1^2 x_2$	$x_1^2 x_2^2$	$x_1^2 x_2^3$	
x_1^3	x_1^3	$x_1^3 x_2$	$x_1^3 x_2^2$	$x_1^3 x_2^3$	
$\vdots$					

The quadratic part can be written as matrix multiplication — a quadratic form

$$b_3x_1^2 + b_4x_2^2 + b_5x_1x_2 = x^T [b_3 \ b_5/2; b_5/2 \ b_4] x$$

- This is the quadratic form covered later in this book; cross terms are its early sign

SUMMARY

What Have We Seen from Univariate to Bivariate Polynomial Regression?

Polynomial regression hand-crafts higher powers of x , turning nonlinear fits into linear ones

The design matrix widens with degree; each new column is a Hadamard power of x

Solving uses the same formula as multiple linear regression:
 $b = (X^T X)^{-1} X^T y$

A higher degree lowers training error, but too high a degree causes overfitting

Bivariate polynomial regression adds a cross term $x_1 x_2$ to capture interaction

The quadratic part can be written as a quadratic form $x^T A x$, a preview of later chapters

Key Terms in This Section

Polynomial Regression *Fits nonlinear relationships by introducing higher powers of x*

Element-wise / Hadamard Product *Written $x \odot x$, multiplies corresponding entries*

Gram Matrix, Generalized Inverse $X^T X$, and $(X^T X)^{-1} X^T$

Overfitting *Fits training data very well but performs poorly on new data*

Cross Term *Such as $x_1 x_2$, capturing interaction between two variables*

Degree *The highest power in the polynomial, denoted m*

Design Matrix X — columns are 1, x , and its element-wise higher powers

Underfitting *A model too simple to fit either training or test data well*

Generalization *A model's ability to perform well on unseen data*

Quadratic Form $x^T A x$, to be covered in a later chapter of this book

FURTHER STUDY

Further Reading

Review · 10.2 Multiple Regression

Solving for b follows exactly the same steps — read alongside this section

Further Reading · Quadratic Forms

A later chapter systematically covers $x^T A x$ and its geometry

Further Reading · Machine Learning

Figure 4 is adapted from Machine Learning; explore overfitting and regularization

Open-Source Repository

Code and figures for the whole book: <https://github.com/Visualize-ML>

Exercises Q1 – Q9

Q1 Given code generating one-variable regression data, compute b , $\hat{y}$, ϵ for degree-3 polynomial regression in Python.

Q3 Why does setting $m=1$ make univariate polynomial regression reduce to simple linear regression?

Q5 Why is polynomial regression still linear regression? Where exactly does the linearity lie?

Q7 Why does the sum of squared errors $\|\epsilon\|^2$ drop as degree rises from 2 to 3? Does it always drop?

Q9 Why can a degree-8 polynomial pass "perfectly" through every training point? What does this suggest?

Q2 Solve Q1 with `sklearn.preprocessing's PolynomialFeatures` and `sklearn.linear_model's LinearRegression`, and compare results.

Q4 Write the design matrix X for a degree-4 univariate polynomial regression, and explain each column.

Q6 Prove that the residual vector ϵ is orthogonal to every column of the design matrix.

Q8 In your own words, explain underfitting versus overfitting with an everyday analogy for each.

Exercises Q10 – Q17

Q10 Use numpy to build the design matrix X for a degree-5 univariate polynomial regression, and compute its Gram matrix.

Q12 Fit the same data with degrees 1 through 8, and plot sum of squared errors versus degree.

Q14 Build the design matrix for bivariate quadratic polynomial regression, and verify its column count in code.

Q16 Use matplotlib to overlay fitted curves for different degrees on the same plot for comparison.

Q11 Write a Python function `poly_design_matrix(x, m)` that returns the design matrix for degree m .

Q13 Write out the equation for bivariate cubic polynomial regression — which feature columns does it need?

Q15 Verify that $x^T[[b_3, b_5/2], [b_5/2, b_4]]x$ expands to $b_3x_1^2 + b_4x_2^2 + b_5x_1x_2$.

Q17 Write a Python function `mse(y, y_hat)` and use it to compare models of different degrees.

Exercises Q18 – Q25

Q18 Explore: split the data into train and test sets, and compute each degree's error on both.

Q20 Use `sklearn.preprocessing's PolynomialFeatures` for bivariate quadratic features, and print feature names.

Q22 Verify in code that summing the 6 curves in Figure 4 reproduces its bottom combined curve.

Q24 Apply polynomial regression to a real dataset with an obvious nonlinear relationship, fit and visualize it.

Q19 Observe how training and test error change with degree, and try to trace out a U-shaped curve.

Q21 Explore: add L2 regularization (Ridge) and see if it eases overfitting in high-degree models.

Q23 Practice: write a Python function `predict_poly(x_new, b, m)` to predict on new data at degree `m`.

Q25 Preview: the next chapter covers eigenvalue decomposition — how does it relate to the Gram matrix $X^T X$ here?

10 Prompts to Explore with AI

- 1 Explain why polynomial regression is still linear regression — linear in the parameters b , not in x .
- 2 Explain what the Hadamard product $x \odot x$ means in the design matrix, and how it relates to x^2 .
- 3 Use Python to show how the simple linear regression design matrix extends to a degree- m polynomial one.
- 4 Explain which range of model complexity underfitting and overfitting each correspond to.
- 5 Explain why training error usually falls as degree rises, while test error need not.

10 Prompts to Explore with AI

6 Explain the geometric meaning of the cross term x_1x_2 in bivariate polynomial regression.

7 Explain the correspondence between the quadratic form $x^T Ax$ and the quadratic terms here.

8 Explain what sklearn's PolynomialFeatures does, and how it relates to a hand-written design matrix.

9 Explain why regularization, such as ridge regression, can ease overfitting in high-degree models.

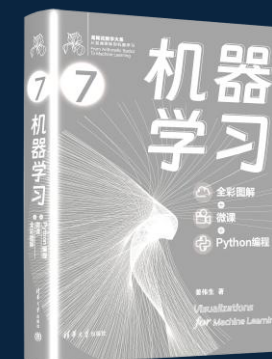
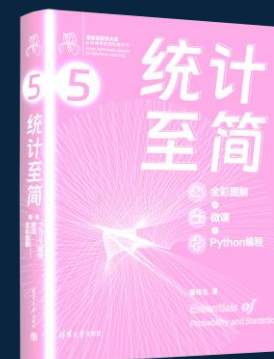
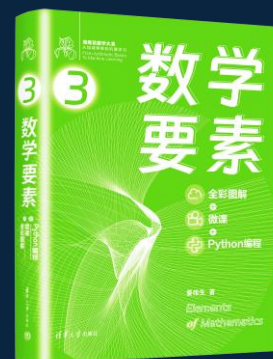
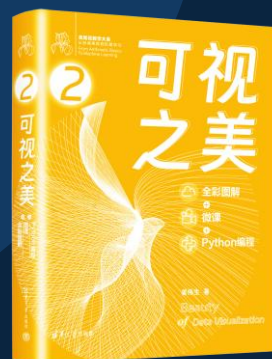
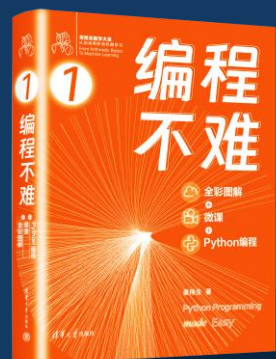
10 Use Python to visualize the same data fit with degree 1, 3, and 8, comparing overfitting behavior.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

